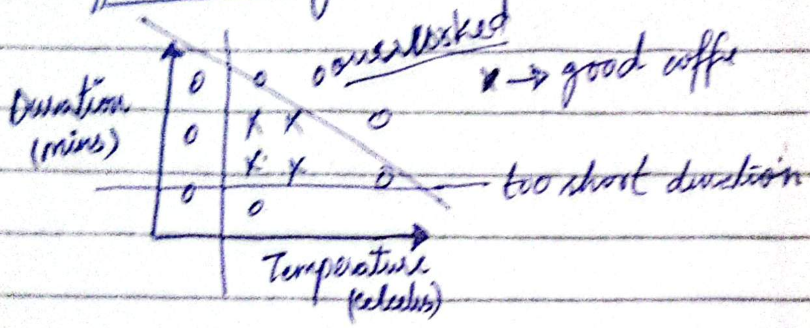
Activation Function

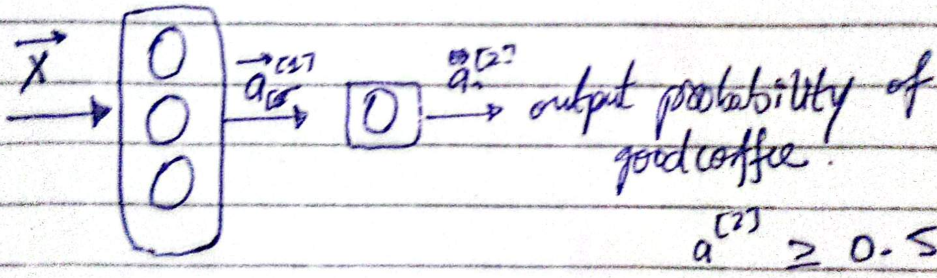
- → What it outputs
- → Why used for
- → ~~what happens when skip~~

Inference

* Forward Propagation

Coffee Roasting





$a^{[2]} \geq 0.5$

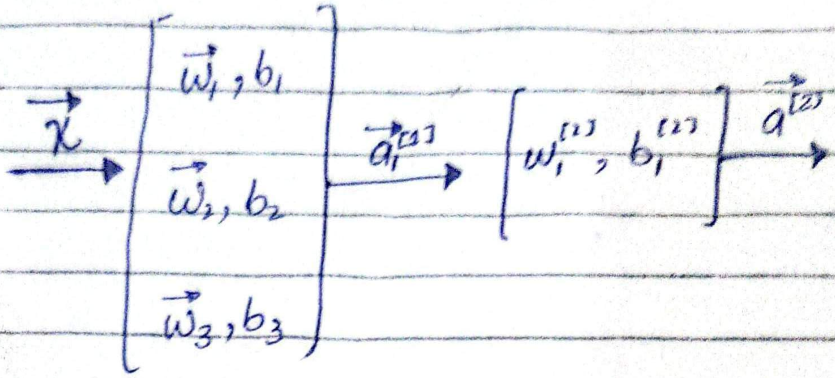
yes → $\hat{y} = 1$

No → $\hat{y} = 0$

Tensorflow implementation:

```
layer1 = Dense(units = 25, activation = "sigmoid")
layer2 = Dense(units = 15, activation = "sigmoid")

model = Sequential([layer 1, layer2])

model.compile( -- )

model.fit()

model.predict(x_new)
```

+ Forward Propagation in a Single Layer:-
(Scratch in python)



x = np.array([200, 17]) → 1-D array

∴ $a_1^{[1]} = g(\vec{w}_1^{[1]} \cdot \vec{x} + b_1^{[1]})$

we need w1_1, b1_1, $z = \vec{w} \cdot \vec{x} + b$

```
w1_1 = np.array([1, 2])
b1_1 = np.array([-1])
z1_1 = np.dot(w1_1, x) + b1_1

a1_1 = sigmoid(z1_1)

w1_2 = np.array([-3, 4])
b1_2 = np.array([1])
z1_2 = np.dot(x, w1_2) + b1_2

a1_2 = sigmoid(z2_1)

a1 = np.array([a1_1, a1_2, a1_3])
```

Now a2

$$a^{[2]} = g(\vec{w}_1^{[2]} \cdot \vec{a}_1^{[1]} + b^{[2]})$$

```
w2_1 = np.array([1,1,2])
b2_1 = np.array([3])
z2_1 = np.dot(w2_1, a1) + b2_1

a2_1 = sigmoid(z2_1)
```

* General Implementation of Forward Propagation.

```
def dense(a_in, W, b):

    units = np.shape(W[1])
    a_out = np.zeroes(units)

    for j in range(units):        j (0, 1, 2)
        w = W[:, j]
        z = np.dot(w, a_in) + b[j]
        a_out[j] = g sigmoid(z)

    return a_out
```